



Culverhouse
College of Business
Institute of Data and Analytics

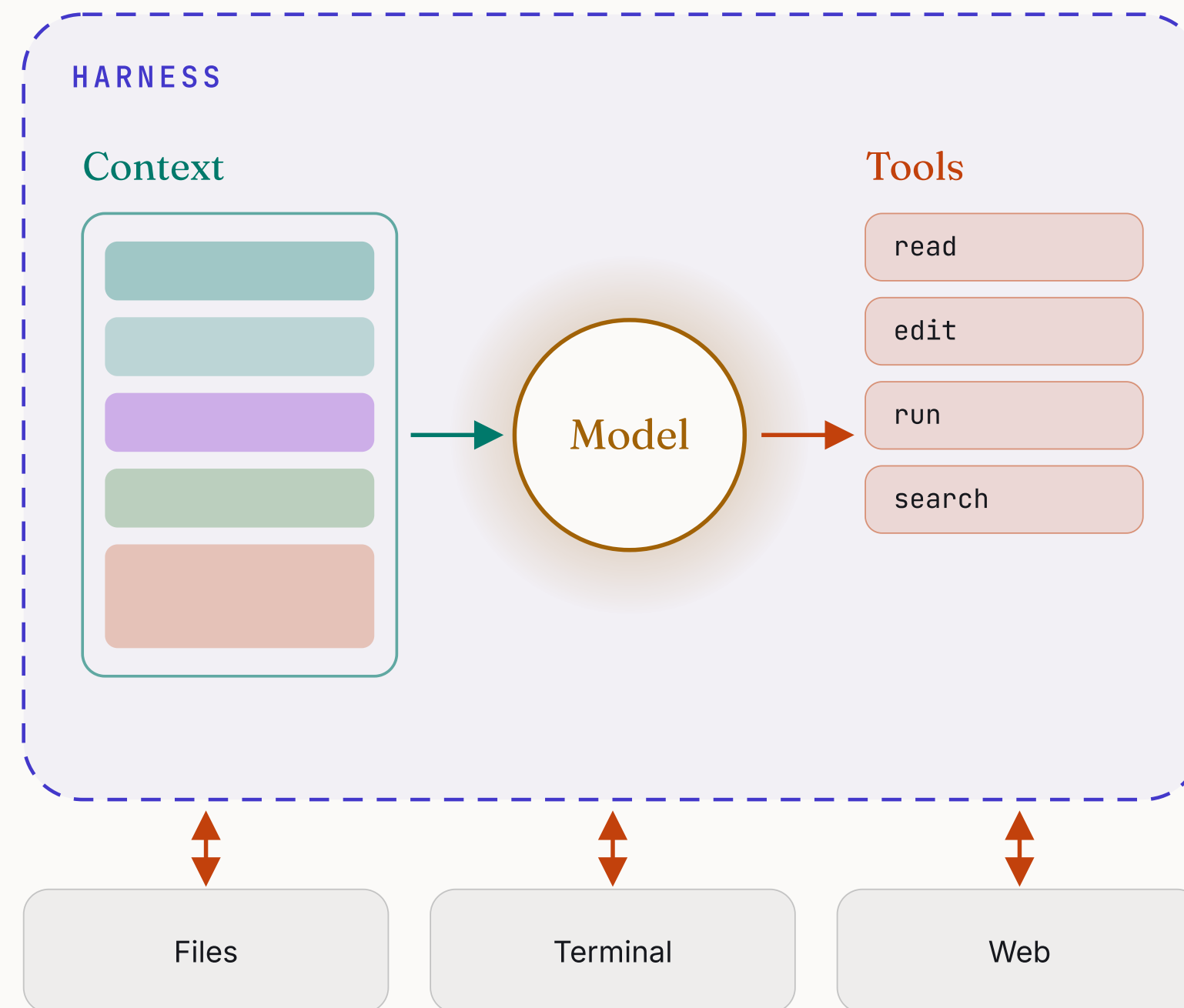
Agentic AI

models that take actions

What makes AI agentic, the parts of an agent, and the vocabulary you will hear.

Nickolas K. Freeman

Institute of Data and Analytics • Culverhouse College of Business
Graduate seminar • September 2026



AI is a field from the 1950s, and LLMs are *one recent part* of it

1956

Artificial intelligence computer systems that do tasks we associate with human intelligence
e.g., chess engines, route planning, and ELIZA (1966), a chatbot built from hand-written rules

1959

Machine learning systems that learn patterns from data instead of following hand-written rules
e.g., spam filters, credit scoring, and product recommendations

2012

Deep learning machine learning with neural networks of many layers
e.g., image recognition, speech-to-text, and machine translation

2020

Large language models deep networks trained on large amounts of text to predict the next token
e.g., GPT, Claude, Gemini, and Llama

The model at the center of most agents today is an LLM.

The same model can *answer a question* or *finish a job*

● CHAT ASSISTANT

My analysis script crashes on the March file. Can you fix it?

This is probably a date-parsing error. Try passing `format="%Y-%m-%d"` to the parser, and check the date column for missing values.

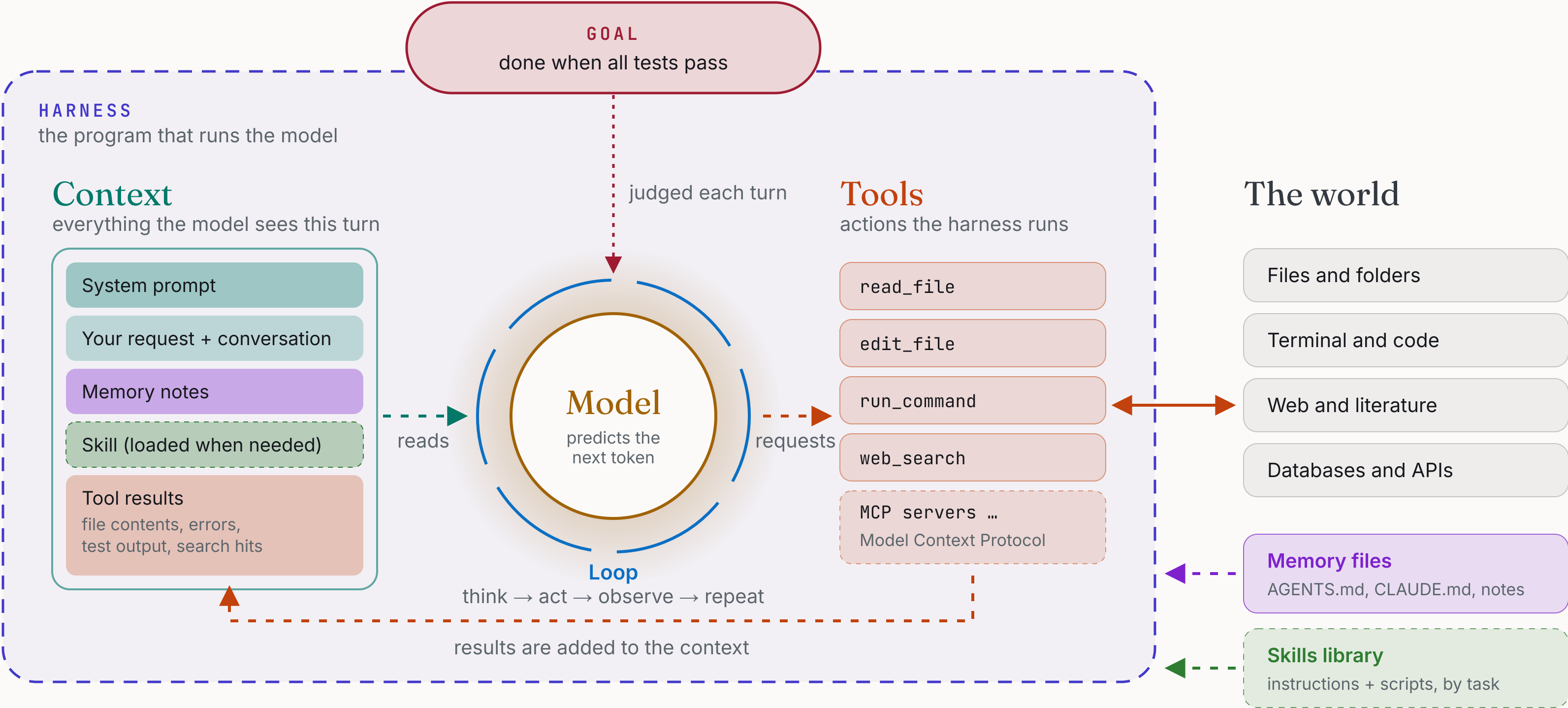
The advice is plausible, but nothing was run or checked, and the fix is still the user's job.

● AGENT

- ▶ `read_file clean.py`
- ▶ `run python clean.py`
`ValueError: time data '03/15/2026'`
- ▶ `read_file data/march.csv (first 20 rows)`
March rows use MM/DD/YYYY
- ▶ `edit clean.py` accept both formats, log rows matching neither
- ▶ `run the automated tests`
`14 passed`
- ✓ Reports the cause, the change, and the test result

Both panels use the same model.

An agent is a system in which a *model chooses its own actions* and takes them through tools



Tools, memory, and skills

TOOLS

Functions the model can request

The model writes a structured request.
The harness runs it and returns the result.

```
{ "tool": "run_command",  
  "input": {  
    "cmd": "pytest tests/" } }
```

MCP (Model Context Protocol) is an open standard, so one tool server works in many harnesses.

MEMORY

Notes that persist between sessions

The model retains nothing between sessions. Memory is text the harness saves and reloads.

```
# AGENTS.md  
- Run the tests before reporting  
- data/raw is read-only  
- March files use MM/DD/YYYY
```

Using an agent does not change the model's weights. Memory is more context, loaded at the start of a session.

SKILLS

Instructions loaded when needed

A folder of instructions and scripts for one kind of task.

```
skills/  
  make-figure/  
    SKILL.md  style.py  
  verify-citations/  
    SKILL.md
```

The agent sees each skill's name and description, and it reads the full instructions only when a task matches.

The prompt is a small part of what the model sees

PROMPT • PROMPT ENGINEERING

The prompt is the instructions. The harness writes a *system prompt*, and the user writes the *user prompt*.

Prompt engineering is crafting the wording, examples, and output format.

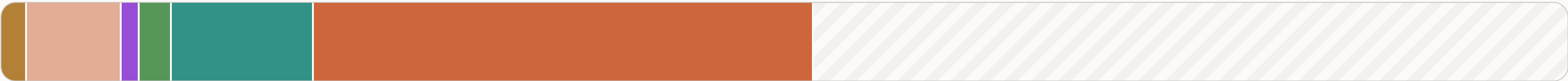
CONTEXT • CONTEXT ENGINEERING

Context is everything the model receives on one call, which may include history, summaries, files, and tool results.

Context engineering is choosing what goes in, what stays out, and when.

An illustrative coding session, partway through (proportions invented for illustration)

200,000-token window

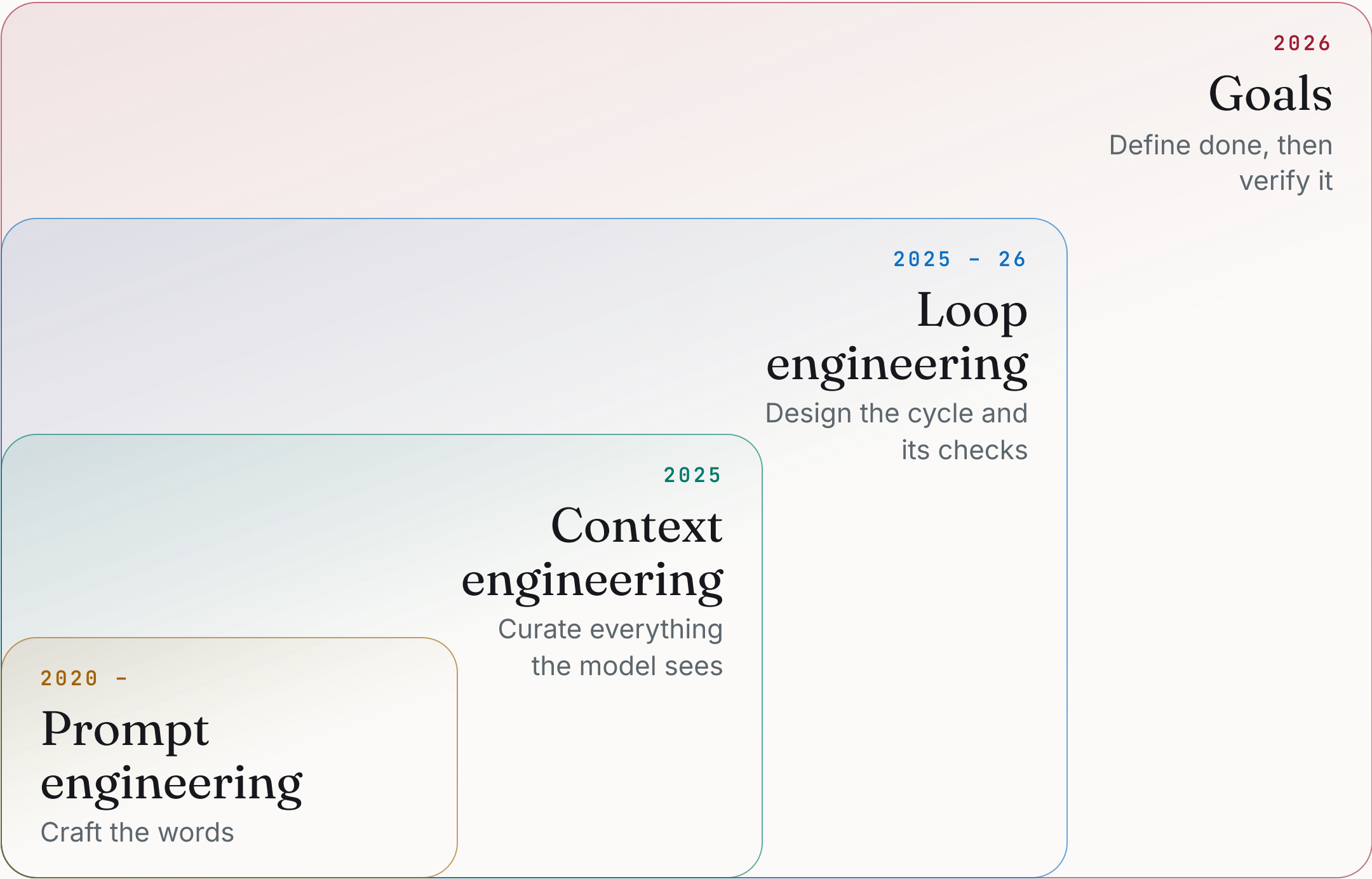


■ System prompt 3k ■ Tool definitions 12k ■ Memory 2k ■ Skill 4k ■ Conversation 18k ■ Tool results 64k □ Free 97k

Context rot

Long or cluttered context can reduce accuracy well before the window is full, and details in the middle are the most likely to be missed. Harnesses respond by summarizing old turns (*compaction*), saving notes to files, and giving subtasks to *subagents*, each of which starts a separate context holding only its instructions and the subtask.

The model does more, and we design at a *higher level*

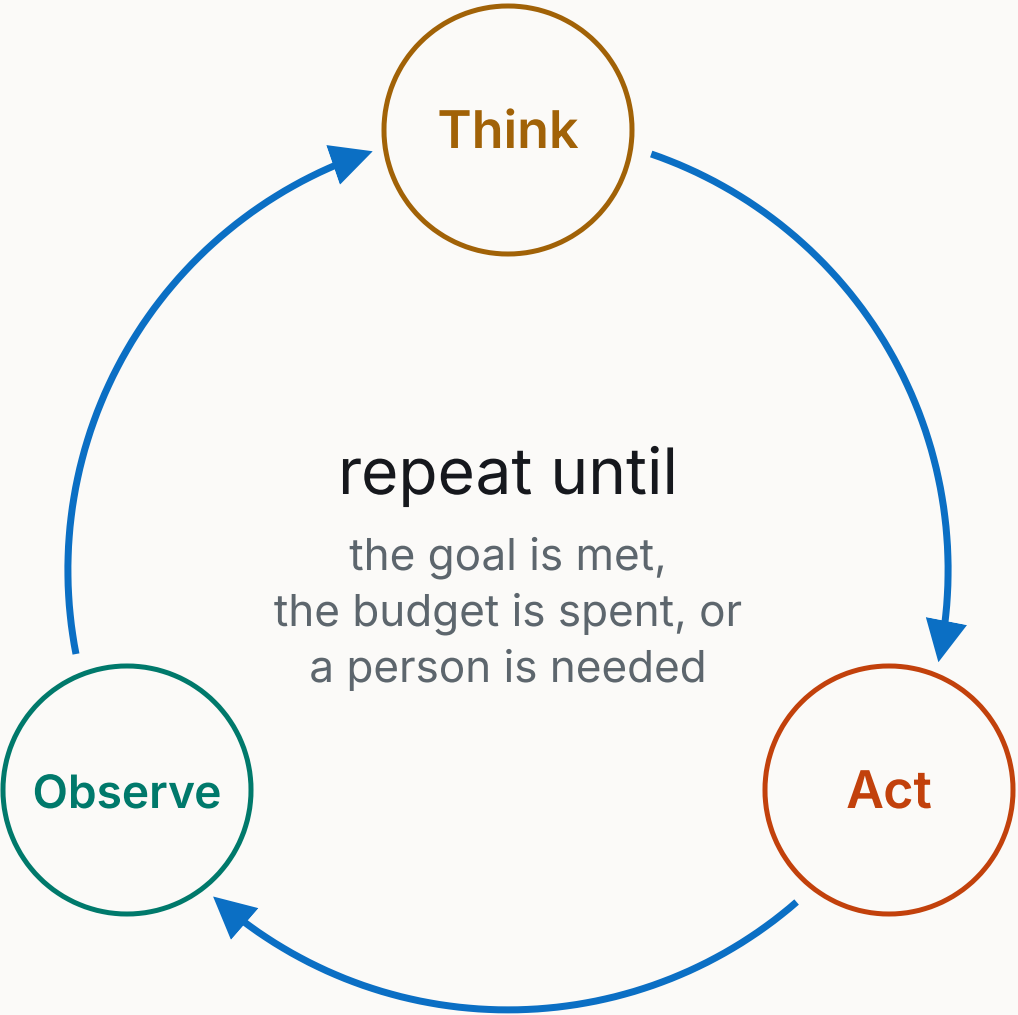


Each level contains the one before it.
Good prompts still matter, and they are now one input among many.

These practices overlap in time. The diagram groups responsibilities rather than dating when each idea began.

"Context engineering" spread as a term in mid-2025. "Loop engineering" and goal modes are 2026 vocabulary, and usage varies.

The loop repeats until a checkable goal is met



• Loop engineering

Designing the cycle the agent repeats

- ◆ **Feedback** it can use, such as tests, validators, compiler errors, or a reviewer model
- ◆ **Stop rules**, i.e., a success condition plus a step or cost budget
- ◆ **Checkpoints** where it pauses for a person
- ◆ **Fresh starts** by splitting long work across subagents

• Goals

VAGUE

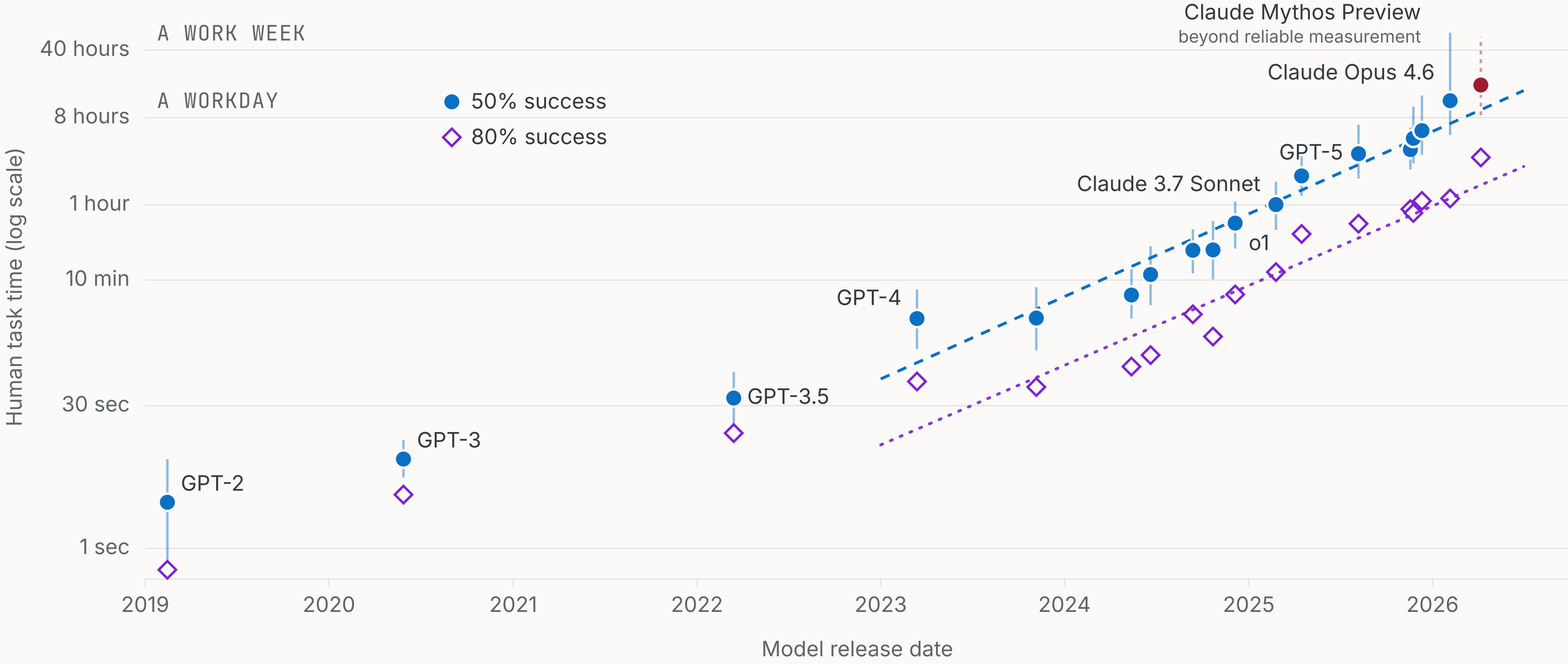
"Clean up the survey analysis."

CHECKABLE

"All 42 tests pass, Figure 3 regenerates from the script, and Table 2 matches its output."

Goal modes (2026) keep an agent working across turns. Claude Code asks a separate model to judge the condition from the conversation, and Codex judges its own work. Either judgment still needs evidence and review.

The tasks agents can finish keep getting longer



Data: METR Time Horizon 1.1 (retrieved 19 September 2026).

Definitions

Agent

A system in which a model chooses its own actions and takes them through tools

Model

The neural network. Tokens in, tokens out, nothing retained between calls

Token

A word piece, the unit models read, write, and bill by

Harness

The program around the model that runs the loop, tools, and permissions

Prompt

The instructions given to the model (system and user)

Prompt engineering

Crafting the wording, examples, and output format

Context

Everything the model receives on one call

Context engineering

Choosing what enters the context, what stays out, and when

Tools

Functions the model requests and the harness executes

MCP

The Model Context Protocol, an open standard for connecting tools to harnesses that support it

Memory

Saved notes reloaded into context in later sessions

Skills

Packaged instructions and scripts, loaded only when relevant

Loop

Think, act, observe, repeat

Loop engineering

Designing the feedback, stop rules, and checkpoints of the loop

Goal

A checkable condition that defines done

Subagent

A helper agent given one subtask, with a context of its own



Culverhouse
College of Business
Institute of Data and Analytics

Questions

Nickolas K. Freeman

Institute of Data and Analytics • Culverhouse College of Business

Graduate seminar • September 2026